

Face Morphing - Final Project Report

1. Method Used

We implement a classical landmark-based face morphing pipeline (Beier-Neely / Delaunay approach, widely studied in face-morphing-attack literature such as Ferrara et al., IJCB 2014). Given two bona fide face photos of different identities (Subject A and Subject B) from Labeled Faces in the Wild (LFW), the pipeline detects dense facial landmarks, aligns corresponding points, computes a Delaunay triangulation on the averaged target shape, performs piecewise-affine warping on every triangle patch, and cross-dissolves the pixel intensities.

src/morph.py - Core pipeline overview:

```
def morph_images(img1, pts1, img2, pts2, alpha, triangles=None):
    pts_morph = (1 - alpha) * pts1 + alpha * pts2 # Averaged target shape
    triangles = delaunay_triangulation(pts_morph, (w, h))
    # Warp each triangle from img1 & img2 into the target shape, then blend:
    morphed = (1 - alpha) * warped1 + alpha * warped2
```

2. Landmark Alignment Strategy

We obtain 478 3D facial landmarks using MediaPipe FaceLandmarker (with 3x bicubic upscaling of the 125x94 LFW crops for robust tracking). Because MediaPipe tracks only the face mesh, we append 8 fixed boundary points (the 4 corners + 4 edge midpoints of the frame) to both landmark arrays. This ensures that background and frame borders participate in the continuous Delaunay triangulation and warping instead of shearing. Correspondence is purely index-based (point i on Face A corresponds to point i on Face B), and one single Delaunay topology computed on the convex combination shape is valid simultaneously for both faces.

src/landmarks.py - Boundary extension & correspondence:

```
def get_landmarks(img_bgr, upscale=3):
    # ... MediaPipe detection of 478 face landmarks ...
    pts = np.array([[lm.x * w, lm.y * h] for lm in result.face_landmarks[0]])
    # Append 8 boundary points so the full frame participates in the warp:
    pts = np.vstack([pts, _boundary_points(w, h)])
    return pts, img_bgr
```

3. Interpolation Technique

We combine geometric and photometric interpolation. For each Delaunay triangle, we compute the affine transform matrix mapping source vertices to target vertices (`cv2.getAffineTransform`) and warp the source patch (`cv2.warpAffine`). A filled polygon mask composites the warped patch cleanly into the target canvas. The two warped images (`warped1` and `warped2`) are then linearly cross-dissolved with weights $(1-\alpha, \alpha)$.

src/morph.py - Piecewise-affine triangle warp & compositing:

```
def _warp_triangle(src_img, dst_img, tri_src, tri_dst):
    mat = cv2.getAffineTransform(np.float32(tri_src_rect), np.float32(tri_dst_rect))
    warped = cv2.warpAffine(src_patch, mat, (r_dst[2], r_dst[3]), flags=cv2.INTER_LINEAR)
    cv2.fillConvexPoly(mask, np.int32(tri_dst_rect), (1.0, 1.0, 1.0), cv2.LINE_AA)
    dst_slice[:] = dst_slice * (1 - mask_c) + warped_c * mask_c
```

4. Observations and Limitations

- At $\alpha=0.5$, morphs successfully blend both identities into a convincing, single face (demonstrating vulnerability to morphing attacks).
- Mismatched pose/expression across pairs (e.g., open mouth with teeth vs. closed lips) creates visible ghosting or double lips.
- Accessories unique to one subject (such as eyeglasses or hair bangs) fade semi-transparently rather than retaining solid physical structure.
- We use linear cross-dissolve rather than Poisson/gradient-domain cloning, meaning lighting and skin-tone differences are not color-corrected across the subjects.

5. Showcase: 5 Pairs of Original Faces and Morphs ($\alpha=0.5$)



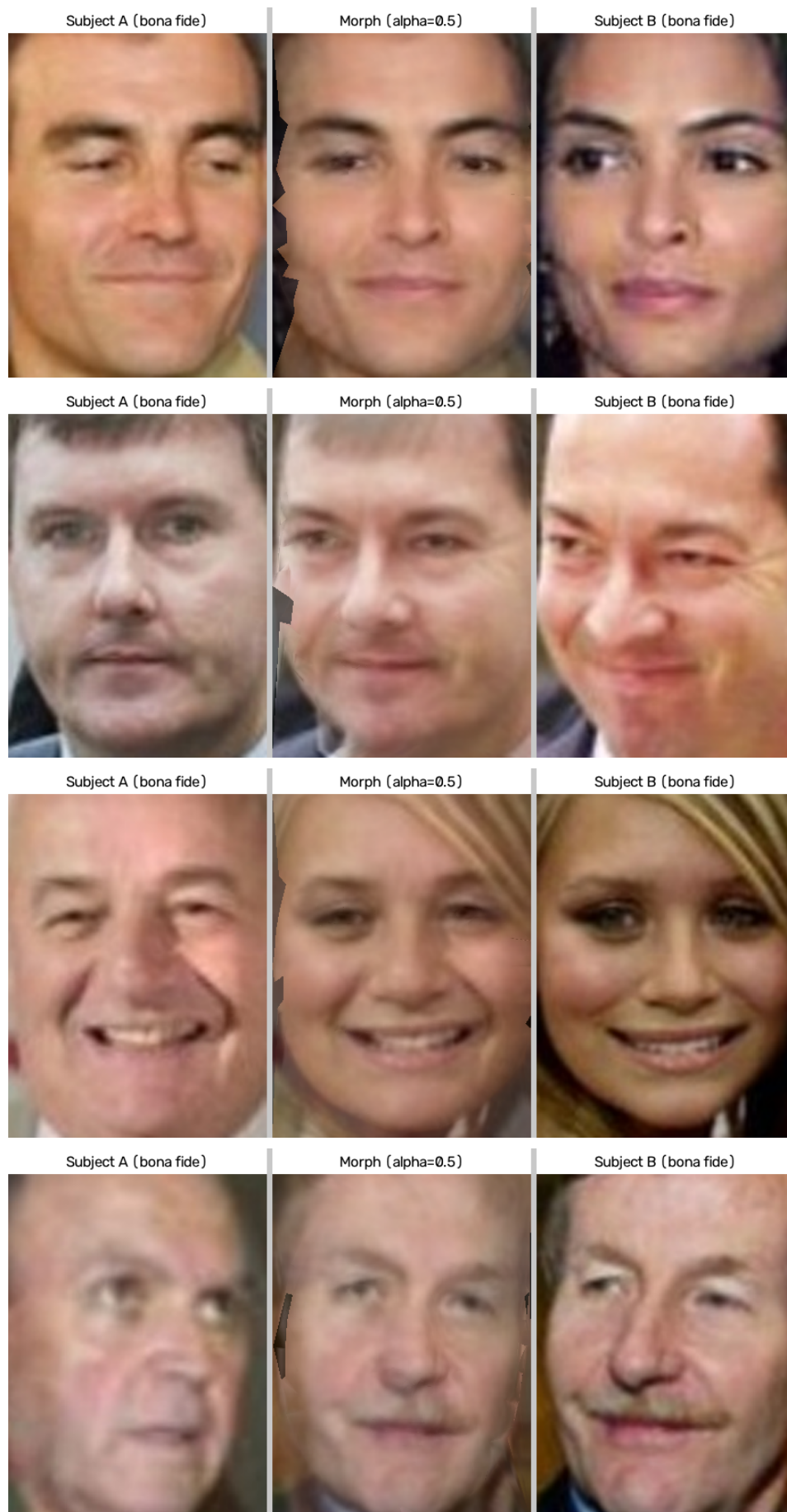


Figure 1: Five complete triptychs of original bona fide face pairs from LFW ('pair000' on Page 1; 'pair003', 'pair004', 'pair007', and 'pair009' above) and their resulting attack morphs at $\alpha=0.5$. Each morph seamlessly fuses the facial geometry and appearance of Subject A and Subject B using our Delaunay triangulation and piecewise-affine pipeline.